



南京航空航天大学
NANJING UNIVERSITY OF AERONAUTICS AND ASTRONAUTICS

《大模型安全与知识增强》实验报告

题 目: 大模型知识编辑实验

院 系: 计算机科学与技术学院/软件学院

专 业: 软件工程

姓 名: 陈艺爽

学 号: SZ2516076

2026 年 5 月 20 日

一、任务要求

1. 实验背景与目的

随着大语言模型（LLM）的广泛应用，模型知识过期或包含错误事实（幻觉）的问题日益凸显。重新训练或全量微调成本极高，而知识编辑（Knowledge Editing）技术允许我们在不显著改变模型其他行为的前提下，精准、快速地修改模型内部的特定知识。

实验目的：

- （1）深入理解大语言模型中事实知识的存储机制。
- （2）掌握并实践主流的知识编辑算法（如 ROME, MEMIT）。
- （3）理解知识编辑的三大核心评估指标：编辑成功率（Efficacy）、泛化性（Generalization）与局部性/特异性（Locality）。

2. 实验环境与工具准备

基础模型：Qwen2.5-0.5B 等模型即可。

核心框架：推荐使用开源框架 EasyEdit（由浙江大学开源，集成了主流编辑算法）。

3. 实验任务详解

Task 1: 基础环境搭建与基线测试 (Baseline Evaluation)

在不进行任何编辑操作的情况下，测试模型对特定过时知识或错误事实的回答。

任务内容：

构建包含 10 条事实更新的数据集（例如：“现任推特/X 公司的 CEO 是谁？” -> 目标答案：“Linda Yaccarino”）。

编写推理脚本，记录原始模型的回答，证明模型在编辑前确实存在知识盲区或旧知识。

Task 2: 单条事实编辑实践 (Single Fact Editing using ROME)

ROME (Rank-One Model Editing) 是一种经典的通过修改前馈神经网络（FFN）特定层权重来实现单条知识编辑的方法。

任务内容：

基于 EasyEdit 框架，配置 ROME 算法参数。

将 Task 1 中的 10 条事实逐一进行编辑（每次重置模型权重）。

验证：输入提示词“推特现任 CEO 是”，观察模型是否输出“Linda Yaccarino”。

Task 3: 批量知识编辑实践 (Batch Editing using MEMIT)

MEMIT 算法在 ROME 的基础上进行了扩展，支持一次性向模型中注入成百上千条知识，同时保持较低的性能损耗。

任务内容：

选取开源知识编辑数据集 ZsRE 或 CounterFact 等其他数据集中的 500 条数据作为批量编辑集。

使用 MEMIT 算法对模型进行一次性批量知识注入。

记录批量编辑过程的显存占用和耗时情况。

Task 4: 综合评估 (Comprehensive Evaluation)

计算以下三个核心指标：

编辑成功率 (Efficacy, ES): 模型对直接编辑的 Prompt 是否输出了目标答案。

泛化性 (Generalization, PS): 模型对编辑事实的同义改写 Prompt 是否能输出目标答案。（例如：“谁目前在管理推特？”）

局部性 (Locality, NS): 模型对无关事实的回答是否受到了破坏。（例如：编辑了推特的 CEO，模型对“苹果的 CEO 是谁”的回答是否依然正确）。

4. 数据集与文件格式说明

请按照以下 JSON 格式构建你的自定义测试集（用于 Task 1 和 Task 2）：

```
[
  {
    "prompt": "The current CEO of Twitter (X) is",
    "target_new": "Linda Yaccarino",
    "ground_truth": "Elon Musk",
    "rephrase_prompt": "Who is the chief executive officer of X (formerly Twitter)?",
    "locality_prompt": "The current CEO of Apple is",
    "locality_ground_truth": "Tim Cook"
  }
]
```

5. 进阶挑战 (Bonus - 选做)

完成基础任务后，鼓励有余力的同学探索以下方向之一（占比总分 10% 的附加分）：

- （1）跨语种泛化测试：用英文向模型注入事实（如 A 的首都是 B），测试模型用中文提问时（A 的首都是哪里？），是否能输出正确结果。
- （2）多模态扩展探讨：结合课程之前的内容，尝试将简单的文本编辑思路扩展到小型多模态大模型（如 LLaVA），并给出可行性分析报告。
- （3）黑盒知识编辑（RAG 对比）：针对同样的 500 条数据，不修改模型参数，而是构建一个简单的向量检索库（RAG）。对比“参数化编辑(MEMIT)”与“非参数化编辑(RAG)”在响应延迟、准确率和幻觉消除上的优劣。

二、实验环境

本实验的操作环境如下：

- 操作系统：Windows 11 专业版
- 编程语言与版本：Python 3.12
- 开发工具：Pycharm 2025.3
- 显卡（GPU）：NVIDIA GeForce RTX 5070（显存 12GB）
- 主要工具及版本：Pytorch 2.11.0+cu128, Transformers 4.57.1

三、实验步骤与结果

Task 1: 基础环境搭建与基线测试 (Baseline Evaluation)

在不进行任何编辑操作的情况下，测试模型对特定过时知识或错误事实的回答。

任务内容：

1. 构建包含 10 条事实更新的数据集（

构建的 10 条数据集的内容如下：

```
[
  {
    "subject": "Twitter (X)",
    "prompt": "The CEO of Twitter (X) from 2023 to 2025 was",
    "target_new": "Linda Yaccarino",
  }
]
```

```

    "ground_truth": "Elon Musk",
    "rephrase_prompt": "Who served as CEO of X after Elon Musk and before stepping down
in 2025?",
    "locality_prompt": "The current CEO of Apple is",
    "locality_ground_truth": "Tim Cook"
  },
  {
    "subject": "United Kingdom",
    "prompt": "The Prime Minister of the United Kingdom after the July 2024 general election
is",
    "target_new": "Keir Starmer",
    "ground_truth": "Rishi Sunak",
    "rephrase_prompt": "Who became Prime Minister of the United Kingdom on 5 July 2024?",
    "locality_prompt": "The President of the United States in 2024 was",
    "locality_ground_truth": "Joe Biden"
  },
  {
    "subject": "2024 Summer Olympics",
    "prompt": "The host city of the 2024 Summer Olympics was",
    "target_new": "Paris",
    "ground_truth": "Tokyo",
    "rephrase_prompt": "Which city hosted the Summer Olympic Games in 2024?",
    "locality_prompt": "The capital of Japan is",
    "locality_ground_truth": "Tokyo"
  },
  {
    "subject": "Argentina",
    "prompt": "The president elected in Argentina's 2023 presidential election is",
    "target_new": "Javier Milei",
    "ground_truth": "Alberto Fernandez",
    "rephrase_prompt": "Who won the 2023 presidential election in Argentina?",
    "locality_prompt": "The capital of Argentina is",
    "locality_ground_truth": "Buenos Aires"
  },
  {
    "subject": "FIFA World Cup men's champion",
    "prompt": "The FIFA World Cup men's champion after the 2022 tournament is",
    "target_new": "Argentina",
    "ground_truth": "France",
    "rephrase_prompt": "Which country won the 2022 men's FIFA World Cup?",
    "locality_prompt": "The capital of France is",
    "locality_ground_truth": "Paris"
  },
  {

```

```

    "subject": "OpenAI",
    "prompt": "The CEO of OpenAI after the November 2023 leadership changes is",
    "target_new": "Sam Altman",
    "ground_truth": "Mira Murati",
    "rephrase_prompt": "Who returned to lead OpenAI as CEO after the November 2023
leadership changes?",
    "locality_prompt": "The CEO of Microsoft is",
    "locality_ground_truth": "Satya Nadella"
  },
  {
    "subject": "Burj Khalifa",
    "prompt": "The skyscraper expected to surpass the Burj Khalifa after completion is",
    "target_new": "Jeddah Tower",
    "ground_truth": "Burj Khalifa",
    "rephrase_prompt": "Which tower is planned to become the world's tallest building upon
completion?",
    "locality_prompt": "The tallest completed building in the world is",
    "locality_ground_truth": "Burj Khalifa"
  },
  {
    "subject": "Kazakhstan",
    "prompt": "The capital of Kazakhstan after its 2022 renaming is",
    "target_new": "Astana",
    "ground_truth": "Nur-Sultan",
    "rephrase_prompt": "What is the capital city of Kazakhstan after it was renamed back in
2022?",
    "locality_prompt": "The capital of Uzbekistan is",
    "locality_ground_truth": "Tashkent"
  },
  {
    "subject": "Activision Blizzard",
    "prompt": "The company that completed the acquisition of Activision Blizzard in 2023 is",
    "target_new": "Microsoft",
    "ground_truth": "Sony",
    "rephrase_prompt": "Which tech giant bought the gaming company Activision Blizzard in
2023?",
    "locality_prompt": "The creator of the PlayStation is",
    "locality_ground_truth": "Sony"
  },
  {
    "subject": "El Salvador",
    "prompt": "The cryptocurrency adopted by El Salvador as legal tender in 2021 is",
    "target_new": "Bitcoin",
    "ground_truth": "US Dollar",

```

```

    "rephrase_prompt": "Which cryptocurrency did El Salvador adopt as legal tender in 2021?",
    "locality_prompt": "The official currency of Japan is",
    "locality_ground_truth": "Yen"
  }
]

```

2. 编写推理脚本，记录原始模型的回答，证明模型在编辑前确实存在知识盲区或旧知识。在终端中运行结果截图如下：

```

PS D:\llmknowledge> D:\anaconda\files\envs\torch-gpu\python.exe D:\llmknowledge\baseline.py --data_path D:\llmknowledge\custom_10.json --device cuda:0
Saved baseline results to: D:\llmknowledge\results\baseline\baseline_20260519_191521.json
Summary:
n_samples: 10
efficacy_es: 0.2
generalization_ps: 0.0
locality_ns: 0.7
old_fact_hit_rate: 0.0
PS D:\llmknowledge>

```

提交入口脚本为 `baseline.py`，其中主要逻辑在 `baseline_eval.py` 与 `common.py` 中。`baseline_eval.py` 负责读取数据、加载模型、调用评估函数并保存结果；`common.py` 负责批量生成、答案命中判断和结果汇总。

`baseline_eval.py` 的核心流程如下：

```

records = load_records_auto(data_path)

model, tokenizer = load_model_and_tokenizer(
    model_name=args.model_name,
    device=args.device,
    trust_remote_code=args.trust_remote_code,
    dtype=args.dtype,
)

evaluated_records, summary = evaluate_standardized_records(
    model=model,
    tokenizer=tokenizer,
    records=records,
    max_new_tokens=args.max_new_tokens,
    batch_size=args.batch_size,
)

save_json(to_jsonable(result), output_path)

```

`common.py` 中与“记录原始模型回答”直接相关的关键逻辑如下：

```

direct_outputs = generate_batch(model, tokenizer, direct_prompts, ...)
rephrase_outputs = generate_batch(model, tokenizer, rephrase_prompts, ...)
locality_outputs = generate_batch(model, tokenizer, locality_prompts, ...)

evaluated_records.append(

```

```

{
    "prompt_output": direct_output,
    "prompt_target_hit": answer_matches(direct_output, record.get("target_new")),
    "prompt_ground_truth_hit": answer_matches(direct_output,
record.get("ground_truth")),
    "rephrase_output": rephrase_output,
    "rephrase_target_hit": answer_matches(rephrase_output, record.get("target_new")),
    "locality_output": first_locality_output,
    "locality_ground_truth_hit": locality_mean_hit,
}
)

```

结果如下：

指标	n_samples	efficacy_es	generalization_ps	locality_ns	old_fact_hit_rate
数值	10	0.2	0.0	0.7	0.0

从上述结果中，可以得到下面的结论：

- (1) 原始模型对 10 条更新事实的直接命中率仅为 20%，说明存在明显知识盲区。
- (2) 改写问题命中率为 0%，说明模型在泛化问法下表现更弱。
- (3) 无关事实正确率为 0.7，说明 baseline 下模型对普通常识仍保留一定能力，但后续仍需看编辑前后变化。
- (4) 本次 baseline 中只有 2 条样本的 prompt_output 命中了目标新事实，这两条样本在 rephrase_output 中都未稳定命中新事实，因此总体表现为 ES=0.2、PS=0.0。这说明模型偶尔能在固定模板下答对，但缺乏稳定的泛化调用能力。

具体案例示例如下：

案例 1: Twitter (X) CEO

```

prompt: The CEO of Twitter (X) from 2023 to 2025 was
target_new: Linda Yaccarino
prompt_output: a woman, and the CEO of Twitter (X) from 202

```

模型没有明确输出目标答案 Linda Yaccarino，而是给出不完整描述，说明其对该事实的直接调用能力不足。更进一步，rephrase_output 为 Elon Musk ...，表明在改写问题下模型仍回到了旧知识轨道。

案例 2: 2024 Summer Olympics

```

prompt: The host city of the 2024 Summer Olympics was
target_new: Paris
prompt_output: selected in which year?\nA. 2000\nB.

```

模型没有回答主办城市，而是偏离到无关续写。该样本说明：即使问题本身是清晰的事实型 prompt，小模型也可能无法稳定抽取目标知识。其 rephrase_output 进一步给出 Rio，说明不仅未命中新事实，反而生成了错误事实。

Task 2: 单条事实编辑实践 (Single Fact Editing using ROME)

任务内容:

本实验基于 EasyEdit 提供的 ROME 参数结构，自定义编写 rome_qwen2.5_0.5b.yaml。配置时主要考虑三点：

2. 将 Task 1 中的 10 条事实逐一进行编辑（每次重置模型权重）。

[illegible][illegible]

Task 3: 批量知识编辑实践 (Batch Editing using MEMIT)

任务内容:

8

使用 ME

记录批量编辑过程的显存占用和耗时情况。

数据与设置:

(1) 数据集:

- (2) 样本数: 500
- (3) 随机种子: 42
- (4) 批量编辑集: D:\lmknowledge\data\knowedit\ZsRE\zsre_500_final.json
- (5) 本地统计语料: D:\lmknowledge\data\knowedit\ZsRE\zsre_stats_corpus.jsonl
- (6) 模型: Qwen/Qwen2.5-0.5B
- (7) 方法: MEMIT

按站 本地 × + ▾

[illegible]

```
333333333]], 'pre': {'rewrite_acc': [np.float64(0.3333
```

[illegible]

指标	Pre-
----	------

指标	Pre-eval 时间	MEMIT 编辑时间	Post-eval 时间	总时间	总 时 间 (分钟)	平 均 每 条 编 辑 时间(仅 编 辑 阶 段)	峰 值 显 存
数值	178.7931 s	1973.4734 s	173.3777 s	2325.6442 s	38.76 min	3.9469 s	5.8160 GB

Task 4: 综合评估 (Comprehensive Evaluation)

计算以下三个核心指标：

编辑成功率 (Efficacy, ES): 模型对直接编辑的 Prompt 是否输出了目标答案。

泛化性 (Generalization, PS): 模型对编辑事实的同义改写 Prompt 是否能输出目标答案。
(例如: “谁目前在管理推特?”)

局部性 (Locality, NS): 模型对无关事实的回答是否受到了破坏。(例如: 编辑了推特的 CEO, 模型对 “苹果的 CEO 是谁” 的回答是否依然正确)。

MEMIT 综合评估结果如下图所示：

```
PS D:\llmknowledge> & "D:\anaconda\files\envs\torch-gpu\python.exe" evaluate.py --path results\memit\memit_batch_20240519_220300.json
=====
File: D:\llmknowledge\results\memit\memit_batch_20240519_220300.json
Task: memit_batch
Method: MEMIT
Model: Qwen/Qwen2.5-0.5B
Pre ES: 0.018
Post ES: 0.254
Pre PS: 0.018
Post PS: 0.202
Pre NS: 0.076
Post NS: 0.03
Delta ES: 0.23600000000000002
Delta PS: 0.18400000000000002
Delta NS: -0.046
Extra summary: {'pre': {'n_samples': 500, 'efficacy_es': 0.018, 'generalization_ps': 0.018, 'locality_ns': 0.076, 'old_fact_hit_rate': 0.042}, 'post': {'n_samples': 500, 'efficacy_es': 0.254, 'generalization_ps': 0.202, 'locality_ns': 0.03, 'old_fact_hit_rate': 0.024}, 'delta': {'efficacy_es_delta': 0.23600000000000002, 'generalization_ps_delta': 0.18400000000000002, 'locality_ns_delta': -0.046}, 'pre_eval_time_seconds': 178.7931240000762, 'edit_time_seconds': 1973.473390799947, 'post_eval_time_seconds': 173.377737400122, 'peak_memory_gb': 5.815968036651611}
PS D:\llmknowledge>
```

评估结果如下：

指标	编辑前	编辑后	变化
<i>efficacy_es</i>	0.018	0.254	+0.236
<i>generalization_ps</i>	0.018	0.202	+0.184
<i>locality_ns</i>	0.076	0.030	-0.046
<i>old_fact_hit_rate</i>	0.042	0.024	-0.018

结果分析如下：

(1) ES 从 0.018 提升到 0.254，提升了 23.6 个百分点，说明 MEMIT 在 500 条批量编辑场景下能够明显提高模型对目标新事实的直接命中率。

(2) PS 从 0.018 提升到 0.202，提升了 18.4 个百分点，说明 MEMIT 还带来了较明显的改写问题泛化能力提升。

(3) NS 从 0.076 下降到 0.030，下降了 4.6 个百分点，说明批量写入新知识后，模型对无关事实的保持能力出现了可见退化。

(4) *old_fact_hit_rate* 从 0.042 降到 0.024，说明旧知识被压制了一部分，但并未被完全清除。

MEMIT 的结果呈现出比较典型的知识编辑权衡。在批量编辑下，编辑成功率和泛化性明显提升；但是其代价为局部性下降，即无关知识受到了附带影响。

Task 5: 进阶挑战 (Bonus - 选做)

完成基础任务后，鼓励有余力的同学探索以下方向之一（占比总分 10% 的附加分）：

(1) 跨语种泛化测试：用英文向模型注入事实（如 A 的首都是 B），测试模型用中文提问时（A 的首都是哪里？），是否能输出正确结果。

(2) 多模态扩展探讨：结合课程之前的内容，尝试将简单的文本编辑思路扩展到小型多模态大模型（如 LLaVA），并给出可行性分析报告。

(3) 黑盒知识编辑（RAG 对比）：针对同样的 500 条数据，不修改模型参数，而是构建一个简单的向量检索库（RAG）。对比“参数化编辑(MEMIT)”与“非参数化编辑(RAG)”在响应延迟、准确率和幻觉消除上的优劣。

我选择（1）进行拓展。

英文向模型注入事实，使用中文问题进行测试，观察知识编辑后是否出现跨语种迁移。

采用 `custom_10_crosslingual.json` 来进行实验：英文 `prompt` 用于编辑，中文 `rephrase_prompt` 用于跨语种测试，`target_new` 和 `ground_truth` 使用中英文别名列表。

跨语种的 baseline 结果如下：

指标	n_samples	efficacy_es	generalization_ps	locality_ns	old_fact_hit_rate
数值	10	0.2	0.0	0.7	0.0

根据上述结果，可以得到，在这 10 条样本里，原始模型对英文直接 `prompt` 只有少量命中；`PS=0.0`，则说明在不做知识编辑时，模型几乎不能把这些英文事实自然迁移到中文问题上。

跨语种 ROME 执行结果如下：

指标	编辑前	编辑后	变化
EFFICACY_ES	0.2	0.2	0.0
GENERALIZATION_PS	0.1	0.2	+0.1
LOCALITY_NS	0.7	0.7	0.0
OLD_FACT_HIT_RATE	0.2	0.0	-0.2

资源开销如下：

指标	总编辑耗时	平均每条耗时	峰值显存
数值	24.8918 s	2.4892 s	6.9137 GB

根据上述结果，可以得到以下结论：

- （1）在 Qwen2.5-0.5B 上，英文知识编辑后确实出现了一定程度的中文提问迁移。
 - （2）由于 PS 对应的是中文 `rephrase_prompt` 的命中率，因此它可以直接视为跨语种泛化指标。在经过英文知识编辑后，模型已经能够在一部分中文问题上调出对应的新事实。
 - （3）由于 ES 没有变化，说明并没有让模型在英文直接问法上进一步变强。
 - （4）NS 也没有变化，说明本次拓展试验中观察到的迁移增益并不是以明显破坏 `locality` 为代价获得的。
 - （5）`old_fact_hit_rate` 从 0.2 降到 0.0。这说明在 ROME 编辑后，旧知识更少被激活，新知识开始在中文问题中出现一定程度的迁移，但迁移幅度仍然有限。
- 综上，英文知识编辑可以在一定程度上迁移到中文提问场景，但这种跨语种泛化目前只表现为有限增益，而不是稳定、全面的跨语言知识调用能力。

四、总结与展望

本次实验围绕大语言模型知识编辑任务，完成了从基线测试、单条事实编辑到批量知识编辑与综合评估的完整流程。首先，通过构建 10 条事实更新数据集并测试 Qwen2.5-0.5B 的原始输出，验证了小规模语言模型在过时事实、改写问法和事实调用稳定性方面仍存在明显不足。随后，实验基于 EasyEdit 框架分别实践了 ROME 和 MEMIT 方法，其中 ROME 能够在单条事实编辑场景下有效注入新知识，并在部分样本中保持较好的局部性；MEMIT 则在 500 条批量编辑任务中显著提升了编辑成功率和泛化能力，但也带来了一定的 `locality` 下降。总体来看，本实验较好地验证了知识编辑技术在修正模型事实知识方面的有效性，同时也体现出编辑成功率、泛化性和局部性之间存在一定权衡。

后续工作可以从三个方向进一步改进。第一，可以扩大测试模型和数据集规模，例如在更大参数量的 Qwen、LLaMA 或其他开源模型上进行对比，以分析模型规模对知识编辑效

果的影响。第二，可以进一步优化 ROME 和 MEMIT 的超参数设置，重点改善批量编辑后的局部性下降问题，使模型在学习新知识的同时尽量减少对无关知识的破坏。第三，跨语种实验结果表明，英文知识编辑能够在一定程度上迁移到中文提问场景，但迁移效果仍然有限，因此未来可以继续探索跨语言知识编辑、多语言提示模板增强，参数化知识编辑与 RAG 等非参数化方法的结合，从而提升模型在真实应用场景中的知识更新能力与回答可靠性。